

UNIT – 9

Pointers

C Programming (CSC115)
BSc CSIT First Semester (TU)

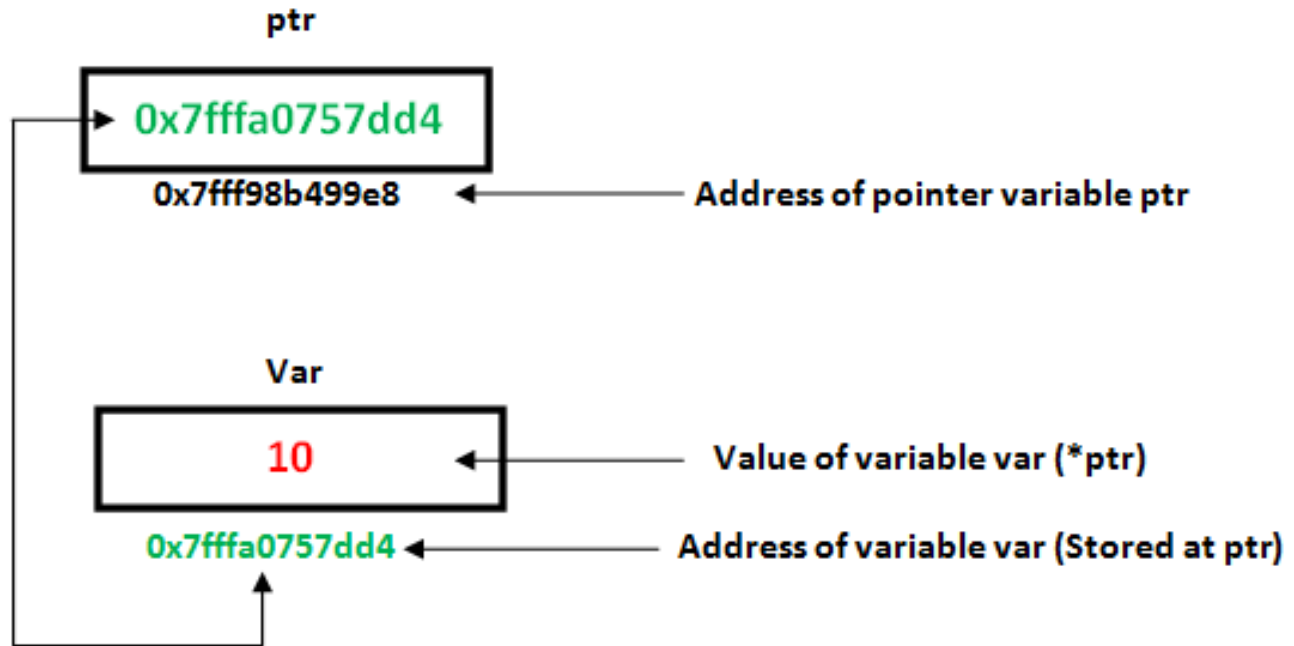
Prepared by:
Dabbal Singh Mahara
ASCOL (2025)

Pointers

- A pointer is a variable that holds address of another variable rather than actual value.
- That is, a pointer is a variable that points to or references a memory location in which data is stored.
- The purpose of pointer is to save memory space and achieve faster execution time.
- Pointers are used frequently in C, as they have a number of useful applications.

Pointer : Example

```
int var = 10;  
int *ptr;  
ptr = &var;
```



Why Pointers?

- Pointers are the mechanism used in C to allow a called function to modify variables in the calling function.
- Pointers provide a way to return multiple data items from a function via function arguments.
- Pointers enable programs to create and manipulate dynamic data structures.
- They produce compact, efficient and powerful code with high execution speed.

The * (asterisk) and &(ampersand) Operators

- When * is used with variable declaration before a variable's name, it becomes a pointer variable, not a normal variable. Eg. `int *p;`
- When * is used in front of pointer variable, it indirectly references the value at that address stored in the pointer. In this case, * is also called indirection or dereference operator.
- The symbol & This is called "referencing" operator.
- This referencing operator is also called address operator, which gives the address of a variable, in which location the variable is resided in the memory.
- Example: `int x = 10;`
`int *p = &x;`

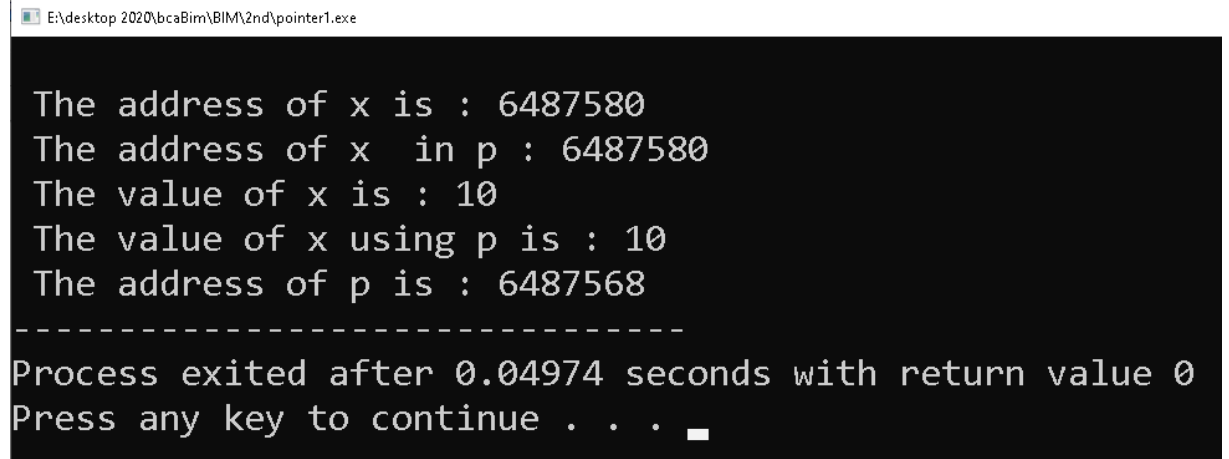
Pointer Declaration and Initialization

- Pointer variables like all other variables, must be declared before they may be used in a C program.
- Its general form is: `data_type *ptrvar;`
- for example: `int *ptr;`
- This statement declares the variable `ptr` as a pointer to `int` i.e. `ptr` can hold address of an integer variable.
- Once the pointer has been declared, it can be initialized to point to a variable using an assignment statement as follows:

```
int a;  
int *x;  
x=&a;
```

Example: Program

```
#include<stdio.h>
int main()
{
    int x = 10;
    int *p = &x;
    printf("\n The address of x is : %u", &x);
    printf("\n The address of x in p : %u", p);
    printf("\n The value of x is : %d", x);
    printf("\n The value of x using p is : %d ", *p);
    printf("\n The address of p is : %u", &p);
    return 0;
}
```

A screenshot of a Windows command prompt window. The title bar shows the file path "E:\desktop 2020\bcaBim\BIM\2nd\pointer1.exe". The window has a black background with white text. The output of the program is displayed as follows:

```
The address of x is : 6487580
The address of x in p : 6487580
The value of x is : 10
The value of x using p is : 10
The address of p is : 6487568
-----
Process exited after 0.04974 seconds with return value 0
Press any key to continue . . .
```

Pointer Arithmetic

- A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value.
- There are four arithmetic operators that can be used on pointers: ++, --, +, and –
- Integer data can be added to or subtracted from the pointer variables.

Incrementing and Decrementing a pointer

- If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing.
- Like increment, we can decrement a pointer variable. If we decrement a pointer, it will start pointing to the previous location.

Example: Pointer Arithmetic

```
#include<stdio.h>
int main(){
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+1;
printf("After increment: Address of p variable is %u \n",p); // in our case, p will get incremented by 4 bytes.
p=p-1;
printf("After decrement: Address of p variable is %u \n",p); // P will now point to the immediate previous location.
p=p+3; //adding 3 to pointer variable
printf("After adding 3: Address of p variable is %u \n",p);

p=p-3; //subtracting 3 from pointer variable
printf("After subtracting 3: Address of p variable is %u \n",p);

return 0;
}
```

Pointers and Array

- An array name by itself is an address, or pointer.
- A pointer variable can take different addresses as values.
- In contrast, an array name is an address, or pointer, that is fixed. That is, array name is a constant pointer. It cannot be assigned any value.
- In case of one dimensional array, an array name is a pointer to the first element in the array.
- Therefore, if X is a 1D array, then X is the address of $X[0]$.
- Moreover, the address of second array element can be expressed as $(X+1)$ or $\&X[1]$.

Pointers and Array

- In general, the address of array element $X[i]$ can be expressed as either $\&X[i]$ or as $(X+i)$.
- Thus, we have two ways to write the address of any array element: either we can write the actual array element preceded by an ampersand; or we can write an expression in which the subscript is added to the array name.
- So, $X[i]$ and $*(X+i)$ both represent the contents of the i th element of X .
- In case of 2D array: `int X[4][5]`
 - $*(p+i)$ gives the address of i th row, to access columns add column number to $*(p+i)$.
 - $(*(x+i) + j)$: It gives the memory address of an element in 2D (i.e. j th element in i th row)
 - $*(*(x+i)+j)$: It gives the value stored in the cell i,j

Pointers and 1D Array

```
// To read n numbers and find average
#include<stdio.h>
#include<conio.h>
#define size 100
int main()
{
    float a[size], sum=0, avg=0;
    int n,i;
    printf("\n How many numbers?\t");
    scanf("%d",&n);
    printf("\n Enter numbers:\n");
    for(i=0;i<n;i++)
    {
        scanf("%f", (a+i));
    }
}
```

```
printf("\n The array elements are:\n");
for(i=0;i<n;i++)
{
    printf("%.1f\t", *(a+i));
    sum = sum +*(a+i);
}

avg=sum/n;
printf("\n Sum =%f\t",sum);
printf("\n Average = %f",avg);

getch();
return 0;
} //end of main
```

Pointers and 2D Array

```
//To add two matrices and print the result in matrix form
#include<stdio.h>
#include<conio.h>
#define ROW 2
#define COL 2

int main()
{
    int a[ROW][COL], b[ROW][COL], i,j, m[ROW][COL];
    printf("\n Enter the elements of first matrix?\n");
    for(i=0;i<ROW;i++)
        for(j=0;j<COL;j++)
            scanf("%d",&*(a+i+j));

    printf("\n Enter numbers:\n");
    for(i=0;i<ROW;i++)
        for(j=0;j<COL;j++)
            scanf("%d",&*(b+i+j));
```

```
for(i=0;i<ROW;i++)
{
    for(j=0;j<COL;j++)
        *(*(sum+i)+j)= *(*(a+i)+j)+*(*(b+i)+j);
}
printf("\n The sum of the matrices:\n");
for(i=0;i<ROW;i++)
{
    for(j=0;j<COL;j++)
    {
        printf("%d\t",*(*(sum+i)+j));
    }
    printf("\n");
}
getch();
return 0;
}
```

Pointers and Character Strings

- A string is a sequence of characters which we save in an array. And in C programming language the `\0` null character marks the end of a string.
- Following example shows creating a string `str` using `char` character array of size 6.

```
char str[6] = "Hello";
```

- The memory representation of this string is as follows:

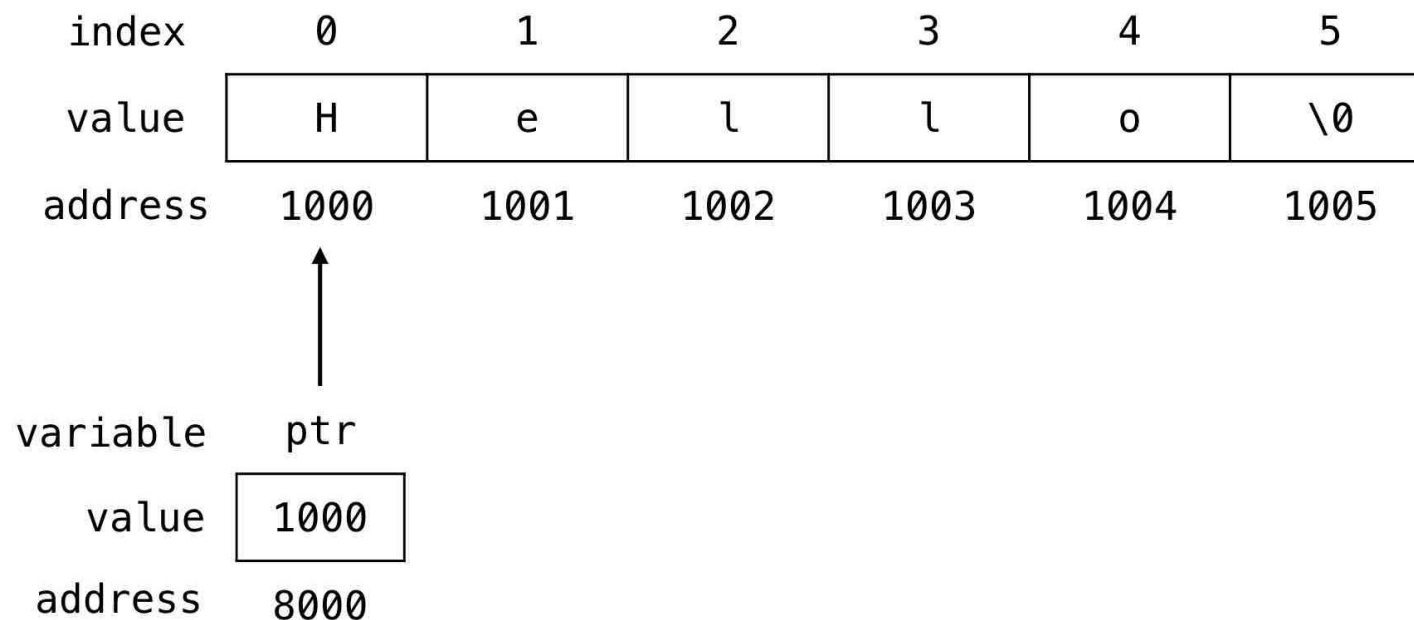
char str[6] = "Hello";						
index	0	1	2	3	4	5
value	H	e	l	l	o	\0
address	1000	1001	1002	1003	1004	1005

Creating a pointer for the string

- The variable name of the string `str` holds the address of the first element of the array i.e., it points at the starting memory address.
- Here, we create a character pointer `ptr` and store the address of the string `str` variable in it. `ptr` will point at the string `str`.

```
char *ptr = str;
```

```
char str[6] = "Hello";
```



Accessing string via pointer

- To access and print the elements of the string we can use a loop and check for the `\0` null character.

```
#include <stdio.h>
int main(void) {
    // string variable
    char str[6] = "Hello";
    // pointer variable
    char *ptr = str;
    // print the string
    while(*ptr != '\0') {
        printf("%c", *ptr);
        // move the ptr pointer to the next memory location
        ptr++;
    }
    return 0;
}
```

Array of Pointers

- An array of pointers is an array in which each cell holds addresses of some data item.
- Following is the declaration of an array of pointers to an integer –
`int *ptr[MAX];`
- It declares ptr as an array of MAX integer pointers. Thus, each element in ptr, holds a pointer to an int value.

Example: Array of Pointers

```
#include <stdio.h>
#define MAX 3

int main () {
    int var[] = {10, 100, 200};
    int i, *ptr[MAX];
    for ( i = 0; i < MAX; i++) {
        ptr[i] = &var[i]; /* assign the address of integer. */
    }
    for ( i = 0; i < MAX; i++) {
        printf("Value of var[%d] = %d\n", i, *ptr[i] );
    }
    return 0;
}
```

Pointer as Argument

- Pointer as a function parameter is used to hold addresses of arguments passed during function call. This is also known as **call by reference**.
- When a function is called by reference any change made to the reference variable will affect the original variable.

Pointer as Argument

```
#include <stdio.h>

void swap(int *a, int *b);

int main()
{
    int m = 10, n = 20;
    printf("m = %d\n", m);
    printf("n = %d\n\n", n);

    swap(&m, &n); //passing address of m and n to the swap function
    printf("After Swapping:\n\n");
    printf("m = %d\n", m);
    printf("n = %d", n);
    return 0;
}
```

```
/*
    pointer 'a' and 'b' holds and
    points to the address of 'm' and 'n'
*/
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

Pointer as Argument : Example 2

```
#include<stdio.h>
void cubebyRef(int *);
int main()
{
    int n=5;
    printf("\n Original value of n = %d",n);
    cubebyRef(&n);
    printf("\nCube = %d",n);
    return 0;
}
```

```
void cubebyRef( int *nptr)
{
    *nptr = (*nptr) * (*nptr)* ( *nptr);
}
```

Functions returning Pointer variables

- A function can also return a pointer to the calling function.
- In this case you must be careful, because local variables of function doesn't live outside the function.
- They have scope only inside the function. Hence if you return a pointer connected to a local variable, that pointer will be pointing to nothing when the function ends.
- So, it is not recommended to return the address of a local variable outside the function as it goes out of scope after function returns.
- **Static Variables** have a property of preserving their value even after they are out of their scope. So to execute the concept of returning a pointer from function in C you must define the local variable as a static variable.

Example: Function Returning pointer

```
#include <stdio.h>
int* larger(int*, int*);
int main()
{
    int a = 15;
    int b = 20;
    int *p;
    p = larger(&a, &b);
    printf("%d is larger", *p);
    return 0;
}
```

```
int* larger(int *x, int *y)
{
    if(*x > *y)
        return x;
    else
        return y;
}
```


C program to illustrate the concept of returning pointer from a function

```
#include <stdio.h>
```

```
// Function returning pointer
```

```
int* fun()
```

```
{
```

```
    int A = 10;
```

```
    return (&A);
```

```
}
```

Note: The function fun() returns pointer to local variable. So, will not print value of A. It will show some garbage value.

```
// main function
```

```
int main()
```

```
{
```

```
    // Declare a pointer
```

```
    int* p;
```

```
    // Function call
```

```
    p = fun();
```

```
    printf("%p\n", p);
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

C program to illustrate the concept of returning pointer from a function

```
#include <stdio.h>
```

```
// Function returning pointer
```

```
int* fun()
```

```
{
```

```
    static int A = 10;
```

```
    return (&A);
```

```
}
```

```
// main function
```

```
int main()
```

```
{
```

```
    // Declare a pointer
```

```
    int* p;
```

```
    // Function call
```

```
    p = fun();
```

```
    printf("%p\n", p);
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

Note: The function fun() returns pointer to local variable. But it is static variable. So, will print value of A as 10.

Dynamic Memory Allocation

- Dynamic memory allocation refers to allocating and freeing memory at execution time or run time.
- It is the ability of program to obtain more memory space at execution time to hold new nodes and release space no longer needed.
- In static memory allocation (normal variables, arrays), memory size must be known before running the program.
- Because in many programs:
 - Don't know the exact size of data in advance
 - The memory requirement may change during execution
 - Static memory allocation is fixed and inflexible
- Dynamic allocation provides flexibility.

Functions for Dynamic Memory Allocation

- There are four library functions malloc(), calloc(), free() and realloc() for memory management.
- These files are defined within header file stdlib.h.

Function	Purpose
malloc()	Allocates a block of memory of a specified size (in bytes).
calloc()	Allocates memory block but initializes to 0
realloc()	Changes size of previously allocated memory
free()	Deallocates memory

Dynamic Memory Allocation

malloc():

- It allocates requested size of bytes and returns a pointer to the first byte of the allocated block.
- Syntax:

`ptr = (data type *) malloc(size_of_block);`

here ptr is pointer of type data_type.

- Eg: `x = ( int *) malloc(100*sizeof(int));`
- Here, a memory space equivalent to 100 times the size of an integer ($100*4=400$ bytes) is reserved and address of first byte of of this memory block is assigned to the pointer x.

Dynamic Memory Allocation

- **calloc():**
 - Unlike malloc() the function calloc() accepts two arguments: no_of_blocks and size_of_each_block.
 - It allocates multiple blocks of memory each of same size and then sets all bytes to zero.
 - syntax: `ptr = (data_type *)calloc(no_of_blocks,size_of_block);`
 - eg. `x= (int*)calloc(10, sizeof(int));`
- **free():** It frees previously allocated space by calloc or malloc or realloc function.
 - The memory dynamically allocated is not returned to the system until the programmer returns the memory explicitly.
 - Syntax: `free(ptr);`
 - *ptr is the pointer to a memory block which has already been created by allocation functions.*

Dynamic Memory Allocation

- **realloc():**
- This function is used to modify the size of previously allocated space.
- syntax: `ptr = realloc( ptr, newsize);`
 - *The first argument ptr is a pointer to a block of memory for which the size is to be altered.*
 - *The second argument newsize specifies the new size of the block.*
- The size may be increased or decreased.
- if newsize is greater than old size and if sufficient space is not available subsequent to the old region, the function `realloc()` creates a new region and all the old data are moved to the new region.

Example 1: Dynamic Memory Allocation for 1D array

```
#include<stdlib.h>
#include<stdio.h>
#include<conio.h>

int main()
{
    int n,i;
    float *p, sum=0,avg;
    printf("\n How many numbers?\t");
    scanf("%d",&n);
    p = ( float *) malloc(n*sizeof(float));
    printf("\n Enter the numbers:\n");
```

```
    for(i=0;i<n;i++)
    {
        scanf("%f",(p+i));
        sum+= *(p+i);
    }
    avg = sum/n;
    printf("\n The sum = %.2f", sum);
    printf("\n The Average = %2.f", avg);
    free(p);
    getch();
    return 0;
}
```


Example 2: Dynamic memory allocation for 2D array

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows, cols;

    printf("Enter rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Step 1: allocate memory for row pointers
    int **arr = (int**) malloc(rows * sizeof(int*));

    // Step 2: allocate memory for each row
    for (int i = 0; i < rows; i++) {
        arr[i] = (int*) malloc(cols * sizeof(int));
    }
```

```
// Input elements
printf("Enter elements:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        scanf("%d", &arr[i][j]);
    }
}

// Display array
printf("2D Array:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}

// Free memory
for (int i = 0; i < rows; i++) {
    free(arr[i]); // free each row
}
free(arr);      // free row pointer array
return 0;
}
```

Example 3: Dynamic memory allocation for structure

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int roll;
    float marks;
};

int main()
{
    struct Student *s;

    s = (struct Student *)malloc(sizeof(struct Student));
```

```
    s->roll = 101;
    s->marks = 88.5;

    printf("Roll = %d, Marks = %.2f", s->roll, s->marks);

    free(s);
    return 0;
}
```

Thank You!